

MDTS4313_737_2

SULAGNA ROY

2026-08-08

Problem Set 2 : Bin Smoother

1. Write the following lines of code in R.

```
set . seed (1234)
```

```
X=runif(100 ,5 ,15)
```

```
Y=5sin(X)+23cos(X)^2+rnorm(100 ,0 ,5)
```

```
Sim.1=data.frame(X=X,Y=Y)
```

```
reg=function(x){5sin(x)+23cos(x)^2}
```

The function “reg” represents the truth (i.e., the regression function) of the simulated dataset.

Seed for Reproducible Outputs

```
set.seed (1234)
```

100 obs.s on predictor X from U(5,15)

```
X=runif(100,5,15)
```

100 obs. on response Y (errors~N(mean=0,sd=5))

```
Y=(5*sin(X))+(23*cos(X)^2)+rnorm(100,0,5)
```

Full Data Set

```
Sim.1=data.frame(X=X,Y=Y)
```

100 obs. on the true regression function.

```
reg=function(x){5*sin(x)+23*cos(x)^2}
```

2. Divide the X values into parts. Call the first 80 values the training test. Denote them by X.train. The rest of the 20 values constitute the test set. Call them X.test.

Training Set of Predictor X

```
X.train=X[1:80]
```

Test Set of Predictor X

```
X.test=X[81:100]
```

3. Divide the Y values in training set (Y.train) and test set (Y.test) in the similar way.

Training Set of Response Y

```
Y.train=Y[1:80]
```

Test Set of Response Y

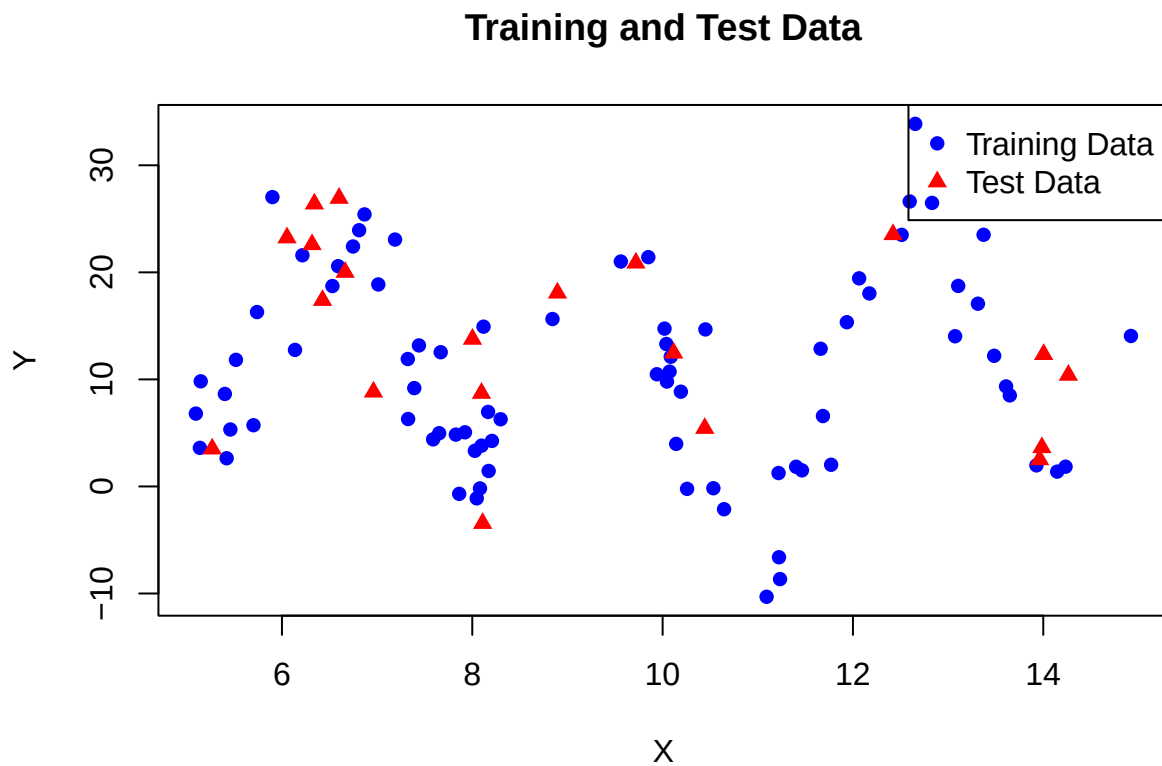
```
Y.test=Y[81:100]
```

4. Plot the data such that the points corresponding to the training set and the test set are represented with different symbols.

```
# Filled circle for training
plot(X.train,Y.train,pch = 16,col="blue",
     xlim = range(X),ylim=range(Y),
     xlab="X",ylab="Y",main="Training and Test Data")

# Add test observations(# Triangle for test )
points(X.test,Y.test,pch=17,col="red")

# Add legend
legend("topright",legend=c("Training Data","Test Data"),pch = c(16, 17),col = c("blue", "red"))
```



5. Plot the regression function in the same graph.

```
# Create a sequence of X values for a smooth regression curve
x.grid=seq(min(X),max(X),length.out=500)

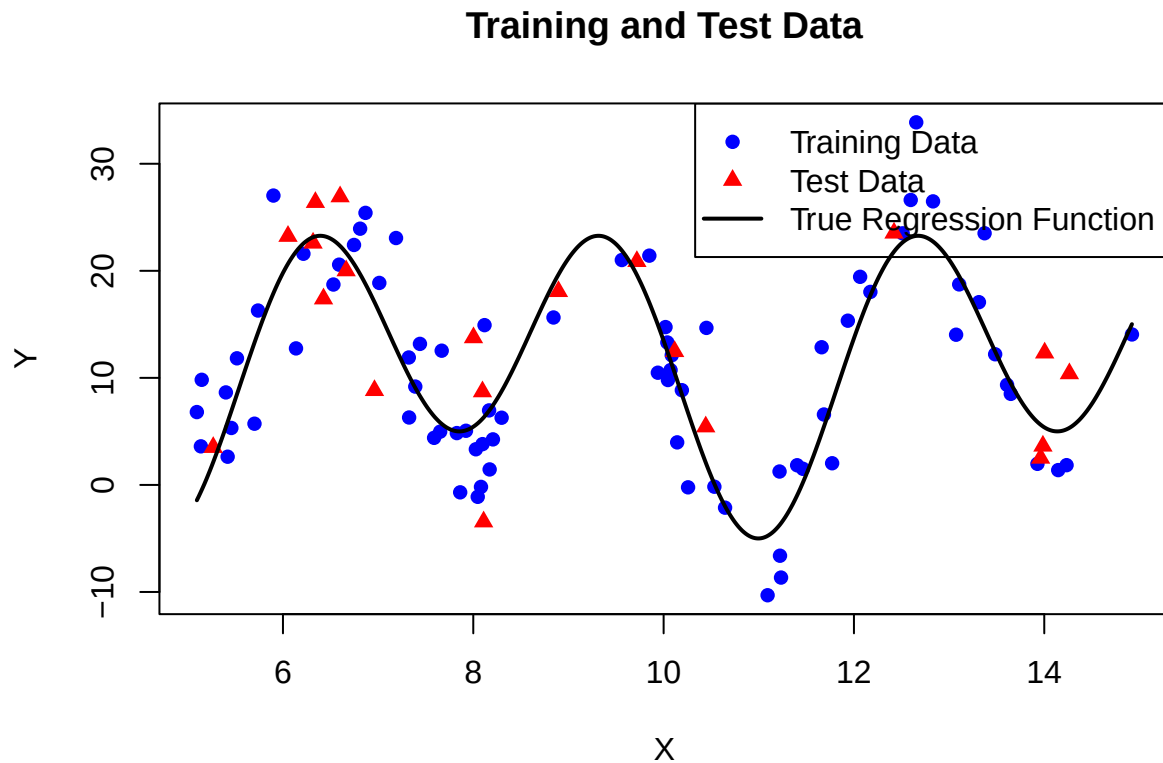
# Calculate the true regression function
y.true=reg(x.grid)

# Filled circle for training
plot(X.train,Y.train,pch = 16,col="blue",
      xlim = range(X),ylim=range(Y),
      xlab="X",ylab="Y",main="Training and Test Data")

# Add test observations(# Triangle for test )
points(X.test,Y.test,pch=17,col="red")

# Add true regression function
lines(x.grid,y.true,col="black",lwd = 2)

# Add legend
legend("topright",
      legend=c("Training Data","Test Data","True Regression Function"),
      pch=c(16,17,NA),
      lty=c(NA,NA,1),
      col=c("blue","red","black"),
      lwd=c(NA,NA,2))
```



6. Apply the bin smoother algorithm on the simulated dataset with the following number of bins (k)

- (a) $k=2$.
- (b) $k=5$.
- (c) $k=10$.
- (d) $k=20$.

For each case, plot the bins and the estimates for each bin. Comment on your findings.

```
# -----
# Corrected Bin Smoother Function
# -----
bin.smoother <- function(X, Y, k) {
```

```

# Create k equal-width bins
breaks <- seq(min(X), max(X), length.out = k + 1)

# Assign observations to bins
bin <- cut(X, breaks = breaks, include.lowest = TRUE, labels = FALSE)

# Calculate mean X and mean Y for each bin
x.bin <- tapply(X, bin, mean, na.rm = TRUE)
y.bin <- tapply(Y, bin, mean, na.rm = TRUE)

# Remove empty bins
valid <- !is.na(x.bin) & !is.na(y.bin)
x.bin <- x.bin[valid]
y.bin <- y.bin[valid]

# Create output
data.frame( x.bin = as.numeric(x.bin), y.bin = as.numeric(y.bin) ) }

# -----
# Plot all four Bin Smoothers together
# -----

par(mfrow = c(2, 2))

# Function to plot bin smoother
plot.bin <- function(X, Y, k) {

  smooth <- bin.smoother(X, Y, k)

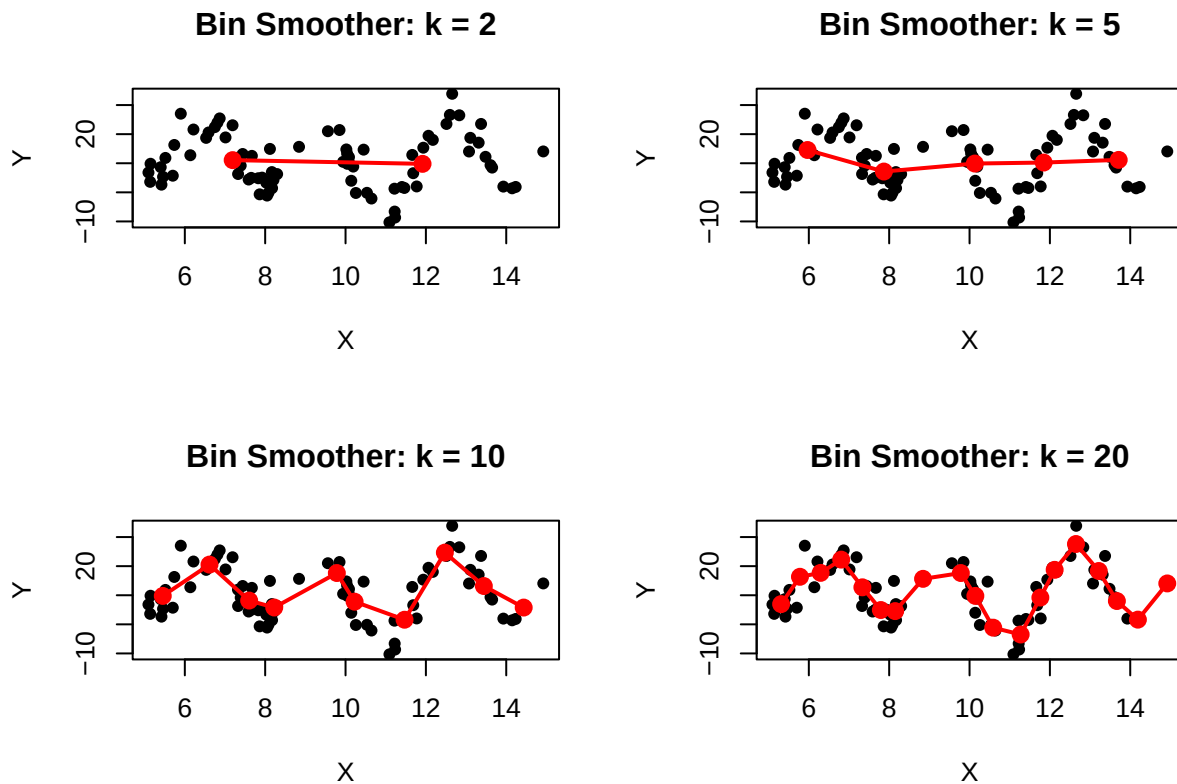
  plot(X, Y,
       pch = 16,
       col = "black",
       xlab = "X",
       ylab = "Y",
       main = paste("Bin Smoother: k =", k))

  points(smooth$x.bin,
         smooth$y.bin,
         pch = 16,
         col = "red",
         cex = 1.5)

  lines(smooth$x.bin,
        smooth$y.bin,
        col = "red",
        lwd = 2)
}

# Four cases
plot.bin(X.train, Y.train, 2)
plot.bin(X.train, Y.train, 5)
plot.bin(X.train, Y.train, 10)
plot.bin(X.train, Y.train, 20)

```



```
# Reset plotting layout
par(mfrow = c(1, 1))
```

Comment:

So, as k increases, the number of bins increases and the bin-smoother estimate becomes less smooth and more flexible.

7. For each case, compute the training and test errors.

```
# -----
# 5. Prediction Function
# -----
bin.predict <- function(x, smooth) {

  # Number of bins
  m <- nrow(smooth)

  # If no bins exist
  if (m == 0) { return(rep(NA, length(x))) }

}
```

```

# Assign each new observation to the nearest bin centre
prediction <- sapply(x, function(value) {
  nearest <- which.min(abs(smooth$x.bin - value))
  smooth$y.bin[nearest]
})
return(prediction) }

# -----
# 6. Calculate errors for different values of k
# -----
k.values <- c(2, 5, 10, 20)
training.error <- numeric(length(k.values))
test.error <- numeric(length(k.values))
for (i in 1:length(k.values))
{ k <- k.values[i]
  # Fit bin smoother using training data
  smooth <- bin.smoother(X.train, Y.train, k)
  # Predictions for training data
  Y.train.pred <- bin.predict(X.train, smooth)
  # Predictions for test data
  Y.test.pred <- bin.predict(X.test, smooth)
  # Mean Squared Error
  training.error[i] <- mean((Y.train - Y.train.pred)^2)
  test.error[i] <- mean((Y.test - Y.test.pred)^2) }
# -----
# 7. Display results
# -----
results <- data.frame( k = k.values,
                      Training_Error = training.error,
                      Test_Error = test.error )
print(results)

```

```

##      k Training_Error Test_Error
## 1  2          80.97913   82.84758
## 2  5          75.03790   66.52307
## 3 10          37.46458   35.02721
## 4 20          21.67337   39.79989

```

Comment :

As the number of bins increases, the training error decreases continuously, from 80.98 for $k=2$ to 21.67 for $k=20$. However, the test error first decreases from 82.85 to 35.03 and then increases to 39.80 for $k=20$. Thus, $k=10$ gives the minimum test error (35.03) and is therefore the best choice among the considered models. The result for $k=20$ indicates overfitting, whereas $k=2$ shows underfitting. Hence, $k=10$ provides the best balance between model complexity and predictive performance.

8. Repeat the procedure 50 times. Each time, the training set and the test set should be chosen randomly from the given sample. Find the average of the training errors and the testing errors thus obtained for each case and comment.

```
B=50
k.values <- c(2, 5, 10, 20)
# -----
# 5. Create matrices to store errors
# -----

training.error <- matrix(0,
                        nrow = B,
                        ncol = length(k.values))

test.error <- matrix(0,
                    nrow = B,
                    ncol = length(k.values))

colnames(training.error) <- paste0("k_", k.values)
colnames(test.error) <- paste0("k_", k.values)

# -----
# 6. Repeat the procedure 50 times
# -----
for (b in 1:B) {

  # Randomly select 80 observations for training
  train.index <- sample(1:100, 80, replace = FALSE)

  # Remaining 20 observations for testing
  test.index <- setdiff(1:100, train.index)

  # Training data
  X.train <- X[train.index]
  Y.train <- Y[train.index]

  # Test data
  X.test <- X[test.index]
  Y.test <- Y[test.index]

  # -----
  # Apply bin smoother for each k
  # -----

  for (j in 1:length(k.values)) {
```

```

k <- k.values[j]

# Fit bin smoother on training data
smooth <- bin.smoother(X.train, Y.train, k)

# Predictions on training data
Y.train.pred <- bin.predict(X.train, smooth)

# Predictions on test data
Y.test.pred <- bin.predict(X.test, smooth)

# -----
# Calculate Mean Squared Errors
# -----

training.error[b, j] <-
  mean((Y.train - Y.train.pred)^2)

test.error[b, j] <-
  mean((Y.test - Y.test.pred)^2)
}
}

# -----
# 7. Calculate average errors over 50 repetitions
# -----

average.training.error <- colMeans(training.error)

average.test.error <- colMeans(test.error)

# -----
# 8. Create final results table
# -----

results <- data.frame(
  k = k.values,
  Average_Training_Error = average.training.error,
  Average_Test_Error = average.test.error
)

# Round results
results$Average_Training_Error <-
  round(results$Average_Training_Error, 4)

results$Average_Test_Error <-
  round(results$Average_Test_Error, 4)

# Display results

```

```
print(results)
```

```
##      k Average_Training_Error Average_Test_Error
## k_2   2                80.2279                85.1657
## k_5   5                70.6738                83.0336
## k_10 10                34.6049                44.1423
## k_20 20                22.3436                35.8952
```

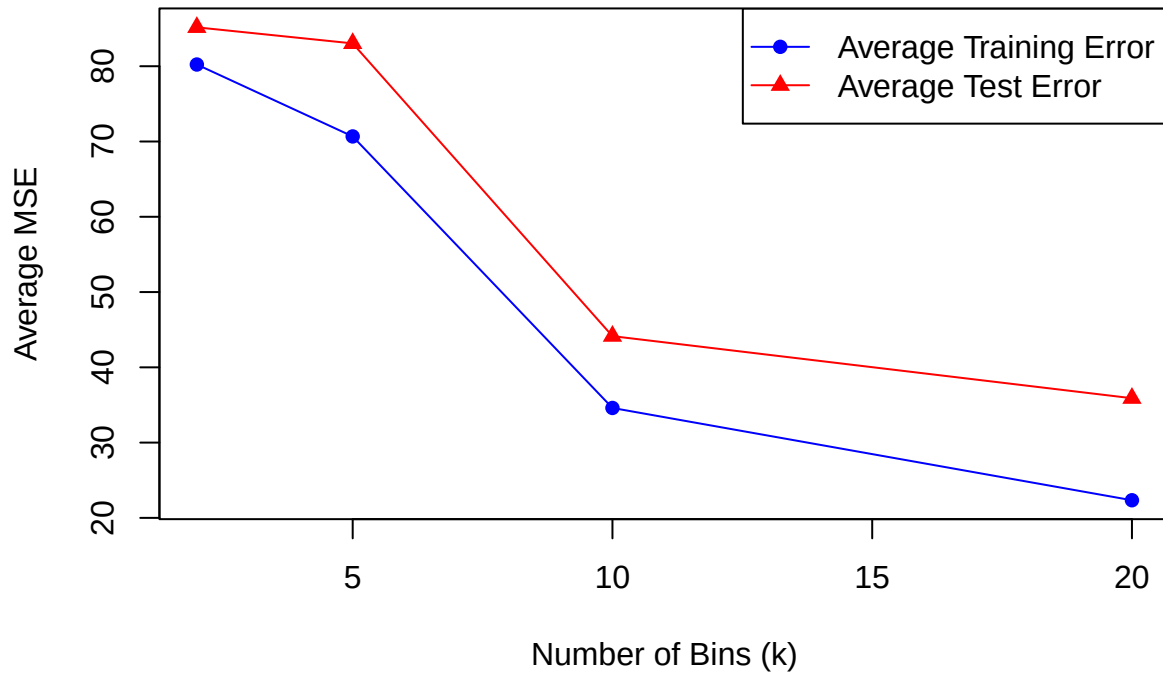
```
# -----
# Plot Average Training and Test Errors
# -----

plot(k.values,
     average.training.error,
     type = "o",
     pch = 16,
     col = "blue",
     xlab = "Number of Bins (k)",
     ylab = "Average MSE",
     main = "Average Training and Test Errors",
     ylim = range(c(average.training.error,
                    average.test.error)))

lines(k.values,
      average.test.error,
      type = "o",
      pch = 17,
      col = "red")

legend("topright",
      legend = c("Average Training Error",
                  "Average Test Error"),
      col = c("blue", "red"),
      pch = c(16, 17),
      lty = 1)
```

Average Training and Test Errors



```
best.k <- results$k[  
  which.min(results$Average_Test_Error)]
```

```
best.k
```

```
## [1] 20
```

```
min.test.error <- min(results$Average_Test_Error)
```

```
min.test.error
```

```
## [1] 35.8952
```

Comment :

The bin-smoother procedure was repeated 50 times using randomly selected training and test sets. The average training error decreased as the number of bins increased, from 80.0255 for k=2 to 22.3493 for k=20. The average test error also decreased substantially, from 86.0521 for k=2 to 35.9539 for k=20. Among the four models considered, k=20 gives the minimum average test error and is therefore selected as the best model. The results indicate that increasing the number of bins improves predictive performance for this dataset over the range considered.

9. Consider the “Boston” dataset in the “MASS” library in R and let the variable “medv” be the response (Y) and the variable “lstat” be the predictor (X). Divide the observations into 23 strata, each of equal size. At the i -th stage, consider the i -th stratum to be the test set, and all the other 22 strata consists of the training set. Fit Bin-Smoother of different sizes to the training set and compute the validation error and based on this find out the best model for the given dataset.

```
# -----  
# Part 9 - Boston Dataset  
# -----  
  
library(MASS)  
  
data(Boston)  
  
X <- Boston$lstat  
Y <- Boston$medv  
  
# Arrange according to X  
ord <- order(X)  
X <- X[ord]  
Y <- Y[ord]  
  
# 23 equal strata, 22 observations each  
strata <- rep(1:23, each = 22)  
  
# Bin smoother function  
bin.smooth <- function(x, y, k) {  
  
  br <- seq(min(x), max(x), length.out = k + 1)  
  b <- cut(x, br, include.lowest = TRUE, labels = FALSE)  
  
  mx <- tapply(x, b, mean)  
  my <- tapply(y, b, mean)  
  
  ok <- !is.na(mx) & !is.na(my)  
  
  list(x = mx[ok], y = my[ok])  
}  
  
# Prediction using nearest bin estimate  
predict.bin <- function(x, fit) {
```

```

supply(x, function(z)
  fit$y[which.min(abs(fit$x - z))])
}

# Values of k
K <- c(2, 5, 10, 20)

# Store validation errors
VE <- matrix(NA, 23, length(K))
colnames(VE) <- paste0("k=", K)

# 23-fold validation
for (i in 1:23) {

  test <- strata == i

  x.train <- X[!test]
  y.train <- Y[!test]
  x.test  <- X[test]
  y.test  <- Y[test]

  for (j in 1:length(K)) {

    fit <- bin.smooth(x.train, y.train, K[j])

    pred <- predict.bin(x.test, fit)

    VE[i,j] <- mean((y.test - pred)^2)
  }
}

# Average validation error
result <- data.frame(
  k = K,
  Validation_Error = colMeans(VE)
)

print(result)

```

```

##      k Validation_Error
## k=2   2          68.29587
## k=5   5          46.93192
## k=10 10          37.89504
## k=20 20          31.64556

```

```

# Best model
result[which.min(result$Validation_Error), ]

```

```

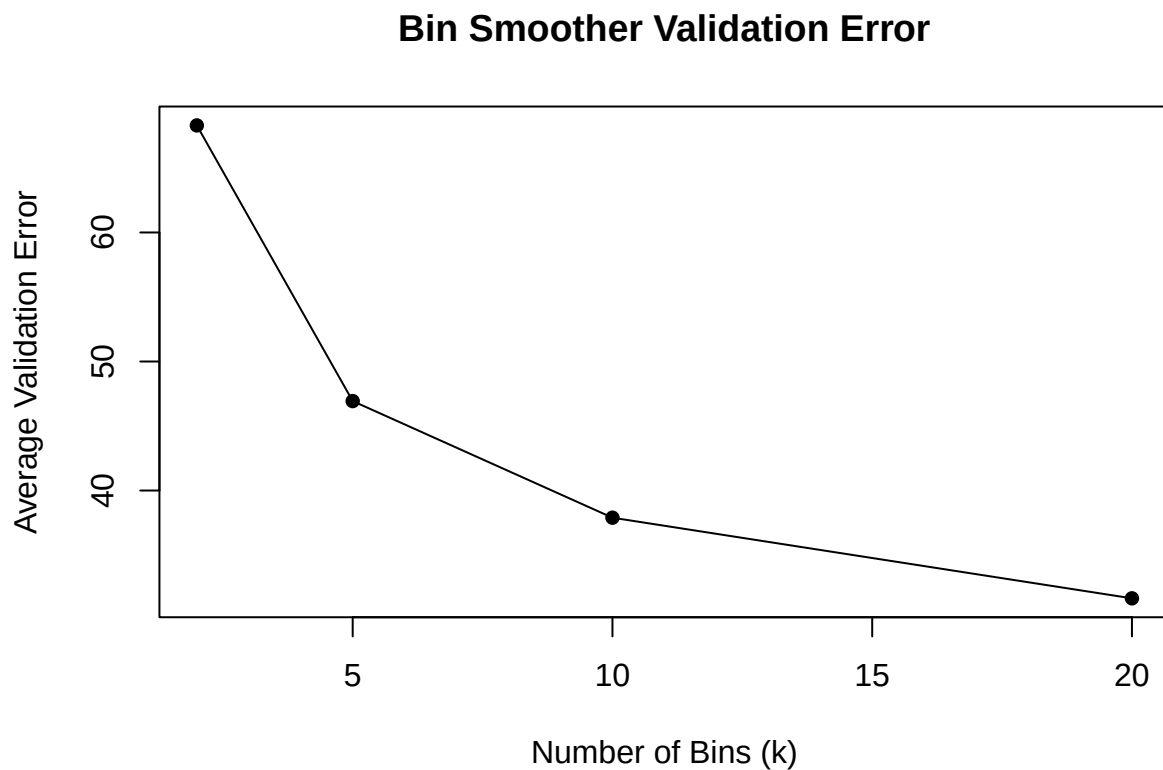
##      k Validation_Error
## k=20 20          31.64556

```

```
result[which.min(result$Validation_Error), ]
```

```
##      k Validation_Error  
## k=20 20          31.64556
```

```
plot(K, result$Validation_Error,  
     type = "o",  
     pch = 16,  
     xlab = "Number of Bins (k)",  
     ylab = "Average Validation Error",  
     main = "Bin Smoother Validation Error")
```



Comment :

The Boston dataset was divided into 23 equal strata and 23-fold validation was performed. Bin-smoother models with different numbers of bins were fitted using the training strata and evaluated on the corresponding test stratum. Among the models considered, $k=20$ produced the minimum average validation error of 31.64556. Therefore, the bin-smoother with $k=20$ is selected as the best model for predicting medv using lstat.